CS 61A — Week 4

Solutions to the handout problems (code for the exercises is in `solutions/`)

P1. `f(2)` prints `down 2` before its recursive call, and `f(1)` prints `down 1` before its call. `f(0)` prints `base` and returns `0`. Now the waiting calls finish, innermost first: `f(1)` computes `0 + 1 = 1` and prints `up 1 1`; `f(2)` computes `1 + 2 = 3` and prints `up 2 3`. Finally the outer `print` displays `3`.

```
down 2
down 1
base
up 1 1
up 2 3
3
```

P2.

1. `sum_a` has no base case. Once `n` reaches `0`, it keeps calling `sum_a(0)` forever, which ends in a `RecursionError`.
2. `sum_b` computes the right value in the recursive case but does not return it, so every call with two or more digits returns `None`.
3. `sum_c` calls itself on the same `n`, not a smaller one, so it never reaches the base case: `RecursionError`.

P3.

```
def count_up(n):
    if n >= 1:
        count_up(n - 1)
        print(n)
```

Leap of faith: assume `count_up(n - 1)` prints $1, \ldots, n-1$ in order. Then printing `n` *after* it gives $1, \ldots, n$. If the `print` came first, `n` would be printed before the smaller numbers and the output would be in decreasing order. For `n = 0` nothing is printed, which is the base case.

P4. The loop has two changing names, `n` and `result`; they become the helper's parameters. The loop condition `n > 0` becomes the base-case test (reversed), and the starting values `n` and `0` are the first call.

```
def reverse(n):
    def helper(n, result):
        if n == 0:
            return result
        return helper(n // 10, result * 10 + n % 10)
    return helper(n, 0)
```

P5. Split by the first move. If it climbs 1 step, $n-1$ steps remain; if 3 steps, $n-3$ remain. Reaching exactly 0 is one way; going below 0 is none.

```
def climb(n):
    if n == 0:
        return 1
    elif n < 0:
        return 0
    return climb(n - 1) + climb(n - 3)
```

Table: `climb(0)`$= 1$, `climb(1)`$= 1$, `climb(2)`$= 1$, `climb(3)`$= 1+1 = 2$, `climb(4)`$= 2+1 = 3$, `climb(5)`$= 3+1 = 4$. The four ways for 5 are 1+1+1+1+1, 1+1+3, 1+3+1, 3+1+1.

P6. Ask what the first letter is.

1. First letter `a`: the rest is any valid string of length $n-1$: `count_strings(n - 1)` ways.
2. First letter `b`: the next letter cannot be `b`, so if $n \geq 2$ it must be `a`; then the rest is any valid string of length $n-2$: `count_strings(n - 2)` ways.
3. Base cases: `count_strings(0) = 1` (the empty string) and `count_strings(1) = 2` (`a`, `b`). The length-1 case is needed separately because "`b` then `a`" needs two letters.

Values: $1, 2, 3, 5, 8$ for lengths 0 to 4. So `count_strings(4) = 8`.